System Design from Zero

High-level design and low-level design. A complete offline guide.

This book teaches both maps of a software system. High-level design (HLD) is the city: machines, roads between them, and what happens when the city is busy or something breaks. Low-level design (LLD) is one building: the classes, the methods, the tables, and the rules inside a program.

You do not need a job, a course, or prior vocabulary. Every term is defined at the moment it is used. Later chapters reuse the words on purpose, so they stick.

How to use this book

1. Read chapters 1 through 6 once, in order. They are the spine. TinyLink, a short-link service, is built in full so you can see every step.
2. Read chapters 7 through 18 as the catalog. Each one is a tool you pick up when a design needs it.
3. Study the worked systems in chapter 19 by covering the page and trying the ten-line script in chapter 20 before you peek.
4. Study the LLD half the same way: rules first, then one drill a day from chapter 28.
5. Keep the checklists and the glossary printed beside you when you practice.

> Assumptions in estimates are labeled as assumptions. Change the assumption and the arithmetic changes. The method stays.

Contents

1. The two maps
2. The HLD method
3. TinyLink, a complete HLD
4. Size math you can do on paper
5. APIs, HTTP, and contracts
6. Data that survives a restart
7. Caching
8. Names, doors, and distance: DNS, load balancers, CDN
9. Copies, consistency, and CAP
10. Sharding
11. Queues and delivery promises
12. Transactions, local and distributed
13. Real-time connections
14. Feeds and fanout
15. More than one city
16. Staying up: timeouts, retries, breakers, limits
17. Security
18. Seeing a running system
19. One program or many
20. Tools: ids, hashing, Bloom filters, search, rate limits
21. Worked high-level designs
22. The LLD method and object rules
23. Patterns
24. Concurrency inside one process
25. Memory structures
26. Testing a design
27. Worked low-level designs
28. Checklists, script, and a study plan
29. Glossary
30. The remaining systems people ask for
31. How HLD and LLD meet
32. Formula sheet

1. The two maps

A system is a set of parts that cooperate to do a job for a user. System design is deciding those parts, the messages between them, the facts they remember, and the behavior when a part is slow or gone.

High-level design answers: which big parts exist, how a request travels, where data lives, how the system grows, and what fails first.

Low-level design answers: inside one program, which classes exist, which method calls which, what a table looks like, and how two actions at the same moment stay correct.

	HLD	LLD
Zoom	City	One building
You draw	Boxes and arrows	Classes, methods, tables
A finished sentence	"Open-link checks the cache, then the database, then returns a redirect."	"`LinkService.open` calls `cache.get`, then `repository.findByCode`."
Done when	Someone else can see traffic, storage, and failure	A programmer can write the classes without guessing the rules

HLD chooses the buildings. LLD designs the rooms of the building you were asked to open. A design is complete only when both are filled for the parts that matter.

The cast, in one sitting

Client. The phone, browser, or other program that starts a request. It draws the screen and sends messages. It does not hold the world's data.

Server. A computer that waits for messages and answers. An **application server** runs your rules. A **stateless** application server stores no user session in its own memory between requests. Any copy can serve any user. The database remembers. The server works and forgets. Stateless servers are easy to multiply: put a new copy behind the door and it is useful immediately.

Request and response. One message out, one message back.

API. The menu of allowed requests. Each item has a name, inputs, and the shape of the answer.

Database. The notebook that still has your facts after a restart. Memory on a server is a whiteboard. Restart wipes it.

Cache. A small, fast shelf of copies in front of a slower notebook. A **hit** means the copy was there. A **miss** means you read the notebook and usually refill the shelf.

Load balancer. The door that spreads arriving requests across many copies of a server.

Replication. Keeping extra copies of data. The **leader** (also called primary) accepts new writes. **Followers** (replicas) copy the leader. **Failover** is a follower becoming the leader after the leader stops.

Shard. A horizontal slice of data on its own machine. Users A–M on one notebook, N–Z on another, is sharding.

Queue. A tray of work to do soon. The user can get an answer before the slow chore finishes. **Workers** empty the tray.

CDN. Content delivery network. Copies of files kept in many cities, close to the user.

Latency. How long one person waits after an action. **Throughput.** How many actions finish per second. **QPS** means queries per second.

Availability. The fraction of time the system answers successfully. Three nines, 99.9%, is about 8.8 hours of silence in a year. Four nines, 99.99%, is about 53 minutes. Five nines is about 5 minutes.

Consistency. After a write succeeds, which readers see that write, and how soon. **Strong consistency** means a read after a successful write sees that write. **Eventual consistency** means readers see it after a delay, and the copies agree if new writes pause.

Authentication. Proof of identity. **Authorization.** Proof this identity may do this action.

Idempotent. Repeating the same request leaves the same result as doing it once.

Five questions every design answers

1. Who talks to whom?
2. What happens on the important action, step by step?
3. What happens when many people do it at once?
4. What happens when a machine dies or a network link splits?
5. What is remembered, for how long, and who is allowed to change it?

2. The HLD method

Use these seven steps in order. The picture is step 5. Drawing boxes before you know the actions produces a poster, not a design.

1. **Requirements.** Functional requirements are actions a user can name. Non-functional requirements are qualities: speed, uptime, durability, privacy, cost. Also write a short out-of-scope list so the design stays bounded.
2. **Size.** Writes per second, reads per second, storage, and sometimes bandwidth. State every assumption.
3. **API.** The menu, including errors.
4. **Data model.** Tables or documents, keys, and the lookup the hot path uses.
5. **Boxes and paths.** The path the user waits on, and the path that runs after the answer.
6. **Deep dive.** The one or two hard mechanisms. For a short-link service, that mechanism is how a unique short code is born.
7. **Failure and growth.** What breaks first, and which single addition removes that pressure.

> Say numbers out loud as guesses. A wrong guess you can revise is useful. A silent guess is invisible.

3. TinyLink, a complete HLD

TinyLink turns a long web address into a short one, such as `tiny.link/aZ3kQ9`. Opening the short one jumps to the long one. This chapter is the reference design. Later chapters point back to it.

Requirements

Functional

- Create a short link from a long address.
- Open a short link and land on the long address.
- Optional custom ending, such as `tiny.link/priya`.
- Optional expiry.
- A click count the owner can read later.

Non-functional

- Open is the hot path. It should return in tens of milliseconds and stay up when creates are failing.
- Create can take longer than open.
- Reads outnumber writes by about 100 to 1.
- Short codes generated by the system are not easy to walk in order if we care that strangers cannot scan neighboring links. Custom names are public on purpose.

Out of scope for version 1: accounts, editing a link, a chart dashboard, and deleting a link from every browser that already saved a jump.

Size

Assumptions: 100 million new links a month. Each link is opened about 100 times. A row is about 500 bytes. Keep data 5 years. A month is about 2.5 million seconds. Peak is about twice the average.

- Writes: 100,000,000 / 2,500,000 ≈ **40 writes per second**. Peak ≈ 80.
- Reads: 40 × 100 ≈ **4,000 reads per second**. Peak ≈ 8,000.
- Rows in 5 years: 100,000,000 × 12 × 5 = **6 billion**.
- Disk: 6 billion × 500 bytes ≈ **3 terabytes**, plus index overhead. Plan for several times that, because indexes and replicas multiply storage.

One database can hold a few terabytes. The pressure is 8,000 redirects a second, so the read path needs a cache and more than one application server. The write path is calm.

API

Action	Call	Inputs	Success
Create	`POST /links`	`longUrl`, optional `customCode`, optional `expiresAt`, `requestToken`	`201` with `code` and `shortUrl`
Open	`GET /{code}`	code in the path	`302` with a `Location` header
Stats	`GET /links/{code}/stats`	code	`200` with `clickCount`

Status **302** means “jump there for this request.” Browsers come back next time, so expiry and click counting still see the visit. Status **301** means “this name permanently means that address.” Browsers may remember and skip you next time. TinyLink uses 302 because expiry and counts need every open to touch the service.

Errors: `400` the address is invalid or too long. `404` unknown code. `409` custom code already taken. `410` the link existed and is expired. `429` the caller is over the rate limit.

Data

```
links
  id            bigint primary key
  code          varchar unique
  long_url      text
  created_at    timestamp
  expires_at    timestamp null
  click_count   bigint
  request_token varchar unique null
```

The hot lookup is by `code`, so that column has a unique index. `request_token` makes create safe to retry: the same token inserts once and returns the same row. The long address is small text, so it lives in the row. A photo product would put the file in object storage and store only a pointer.

City map

```
Browser
  -> Load balancer
    -> Stateless API servers
         -> Cache, key = code
         -> SQL database
         -> Click queue -> count workers -> database
```

Create path. Validate the URL. If the caller supplied a custom code, try to insert it. Otherwise take the next id, encode it, insert the row. Return the short URL. Fill the cache only after the insert commits, or leave the first open to fill it.

Open path. Read cache. On a miss, read the database and fill the cache. If missing, 404. If expired, 410. Otherwise publish a click event and return 302. The person who clicked waits through the cache or database read. The counter update happens after the jump, on a worker.

How a short code is born

Base62 uses 62 symbols: digits, lowercase letters, uppercase letters. Seven characters give 62^7, about 3.5 trillion names. Five years of TinyLink needs about 6 billion names, so 7 characters is enough room.

A database sequence can hand out the next `id`. At higher write speed, each application server reserves a **block**, for example "I own ids 1,000,000 through 1,999,999," and hands those out from memory. When the block is used up, it reserves another. If a server dies mid-block, the unused ids are skipped. Holes are harmless. Duplicate ids are not.

Encoding is repeated division by 62. Remainders become characters. The same id always becomes the same code. Decoding reverses it, which tells you the shard later if you shard by id range.

Custom codes skip the encoder. The unique index on `code` lets one insert win. The loser gets 409.

A hash of the long URL can also make a code. Two different URLs can hash to the same characters. You then check the notebook and retry with more characters. The counter design avoids that collision on the ordinary path, so this design uses the counter. Hashing is a fair choice when you want the same long URL to keep the same short code without a lookup table of ids, and you accept collision handling.

What breaks, and the next box

Pressure	Response
8,000 opens per second	Many stateless servers, plus a cache of code to long URL
Cache process dies	Servers read the database. Slower, still correct
Database leader dies	A follower is promoted
Two custom names collide	Unique index, one winner, 409 for the other
Click counts lag by seconds	Accepted. The queue keeps the redirect fast
Crash after insert, before the response	Client retries with the same request token
One code is suddenly famous	One cache entry serves everyone. Popularity helps
Notebook grows past one machine	Shard by id range. Decode the code to find the shard

4. Size math you can do on paper

Use powers of ten so the arithmetic fits in your head.

Fact	Handy figure
Seconds in a day	about 10^5 (86,400)
Seconds in a month	about 2.5×10^6
Seconds in a year	about 3×10^7
1 million requests a day	about 12 per second
1 million requests a month	well under 1 per second
A character	about 1 byte of text, more if you store rich Unicode details
A million bytes	1 MB
A thousand MB	1 GB
A thousand GB	1 TB
A typical server network card	1 to 10 gigabits per second. Divide by 8 to get bytes

QPS = total actions in the period ÷ seconds in the period. Then multiply by a peak factor, often 2× for a daily product and higher for a product that spikes at a single event.

Storage = rows × bytes per row × replicas × years. Add a factor of 2 or more for indexes and waste. Object storage for video is separate: minutes × bitrate. A 1 Mbps stream for one hour is 1 × 3600 / 8 ≈ 450 MB.

Cache memory = hot keys × value size × overhead. You rarely cache everything. Cache the working set, the slice that is read over and over. If 20% of links receive 80% of opens, size the cache for that 20% plus room to grow.

Bandwidth = QPS × bytes on the wire. Redirects are small. Video is large. A product that moves pictures will hit the network before it hits the CPU.

Write the assumption beside the result. “40 writes/s if we take 100 million creates a month.” When the real number is 10×, you know which box to revisit.

5. APIs, HTTP, and contracts

An API is a promise. Clients depend on the name, the inputs, the outputs, and the errors. Changing a promise without a plan breaks people you cannot see.

HTTP methods

Method	Usual meaning	Safe to retry?
GET	Read. Should not change data	Yes
POST	Create, or run an action	Only if you add an idempotency key
PUT	Replace this resource with this body	Yes, if the same body replaces the same resource
PATCH	Change some fields	Depends on the patch. “Set status to paid” is safe to retry. “Add 1 to count” is not, unless the server deduplicates
DELETE	Remove	Yes. Deleting twice ends with the resource gone

GET, PUT, and DELETE are designed to be idempotent. POST is the one you protect with a **request token** (idempotency key): the server stores the token with the result and returns the original result on a replay.

Status codes worth memorizing

Code	Meaning
200	Success with a body
201	Created
202	Accepted. The work will finish later
204	Success with an empty body
301 / 302	Redirect. 301 may be cached by the client as permanent. 302 is for this request
400	The client sent something invalid
401	You have not proved who you are
403	We know who you are, and you may not do this
404	No such resource
409	Conflict, such as a duplicate name
410	Gone. It used to exist
429	Too many requests
500	The server failed
502 / 503 / 504	A neighbor failed, we are unavailable, or a neighbor timed out

Shapes of a good contract

- Nouns in paths: `/links/{code}`. Verbs belong in methods, or in a clearly named action when HTTP has no natural verb.
- Ids in the path. Filters in the query string: `/messages?conversationId=9&limit=50`.
- One error body shape everywhere: a code, a human message, and a field name when a field is wrong.
- Version the contract when a change would break old clients. A path prefix `/v2` is the blunt tool. Adding optional fields is usually safe. Removing or renaming fields is a break.
- Paginate lists. **Offset pagination** (`page=5&limit=20`) is easy and gets slow and skippy when rows are inserted at the top. **Cursor pagination** returns “give me 20 items older than this id.” It stays fast because it uses the index, and it stays stable while new rows arrive.

Timeouts, from the caller’s side

Every call to another machine has a deadline. A client that waits forever holds a worker on your server, which holds a worker on the next server. Pick a budget for the whole user request, then give each hop a smaller budget. If the hop misses the budget, fail that hop and follow your retry rule.

6. Data that survives a restart

Choose storage from the questions you ask, not from fashion.

SQL databases

Data sits in tables of rows and columns. A **primary key** identifies one row. A **foreign key** is a column that points at a primary key in another table, such as `likes.photo_id` pointing at `photos.id`. The database can refuse a like that points at a missing photo.

SQL is a strong fit when facts must match, when you join them in different combinations, and when a bundle of writes must all stick or all roll back. Users, orders, money, seats, and short links belong here.

Normalization means each fact lives in one place.

- **First normal form:** each column holds one value, not a secret list stuffed into a text field. “tags = cat,dog” makes “find every photo tagged cat” a scan.
- **Second:** every non-key column depends on the whole key, not on a piece of it.
- **Third:** a non-key column depends on the key, not on another non-key column. A user’s city belongs on the user row. Copying the city onto every photo means a move requires updating every photo.

Sometimes you copy a fact on purpose. That copy is **denormalization**. You do it when a join on the hot path is too expensive, and you accept the job of keeping the copy honest.

Indexes

An index is an extra structure that answers a lookup without reading every row. The usual structure is a **B-tree**: ordered, good for equality and for ranges such as “photos newer than Tuesday.” A **hash index** answers equality and does not help ranges.

A **composite index** on `(user_id, created_at)` serves “this user’s newest photos.” It also serves “this user” alone, because `user_id` is the leftmost column. It does not serve “all photos from Tuesday” by itself, because `created_at` is not the leftmost column. This is the **leftmost prefix** rule.

A **covering index** contains every column the query needs, so the database can answer from the index alone.

Indexes speed reads and cost writes, because every insert and update maintains them. Index the lookups you actually run. A table with ten unused indexes pays for those ten on every write.

NoSQL, in three useful shapes

Key-value. One key, one blob. Session data, cache, shopping cart, “feed for user 42.” Extremely fast when the access is always by that key.

Document. A key and a JSON-like document you can query inside. A good fit when one aggregate, such as a product with its variants, is always loaded together, and different products have slightly different fields.

Wide-column. A row key and many columns, spread across machines. A fit for huge write streams such as time-series events and messages keyed by conversation.

These systems often push joins and multi-row transactions up into your code. You pick them when the key-based access and the scale are the point.

Object storage

Photos, video, and backups go in object storage: a giant closet of blobs addressed by a key. The database stores the key, the owner, and the time. You do not put a 4 MB photo inside every SQL row. Object storage is cheap per byte and slow compared with a cache of small metadata. Put a CDN in front when many users download the same blob.

Choosing

You mostly…	Lean toward
Join orders, users, and payments, and need transactions	SQL
Fetch one value by one key at very high QPS	Key-value
Store large files	Object storage, plus SQL for metadata
Append a huge stream of events	A log or wide-column store
Search words inside text	A search index, fed from the source of truth

The **source of truth** is the store whose answer wins when copies disagree. Caches, search indexes, and feed timelines are derived. They can be rebuilt from the source of truth.

7. Caching

A cache holds a copy so the next reader skips a slower store. You add one when the same read happens over and over, and a slightly older copy is acceptable for a defined time.

Where it sits

- **Browser and CDN.** Close to the user. Good for public files and responses that are the same for everyone.
- **Application memory.** Fastest, and private to one server. A second server has a cold copy. Useful for tiny, process-local data.
- **Shared cache** (a system such as Redis or Memcached). Every application server sees the same shelf. This is the usual box in an HLD diagram.

How writes meet the shelf

Style	What happens	You accept
Cache-aside	The application reads the cache. On a miss it reads the database and puts the value in the cache. Writes update the database and delete or replace the cache key.	A brief window where the cache is empty or old. This is the default for TinyLink.
Read-through	The application asks only the cache. On a miss the cache library loads from the database.	The same window, with the load hidden inside the cache client.
Write-through	A write updates the cache and the database before success returns.	Slower writes. The cache matches the database after each successful write.
Write-back	A write updates the cache and returns. A later flush writes the database.	Very fast writes, and lost data if the cache dies before the flush.
Write-around	A write goes to the database, and the cache key is deleted. The next read fills the cache.	The next read is a miss. You avoid filling the cache with data nobody rereads.

Deleting a key on write (invalidation) is safer than updating it in place when several kinds of writes exist. A stale key with a **TTL** (time to live) is a backstop: even if you forget to delete, the copy dies on schedule. TTL alone makes the maximum staleness equal to the TTL. Invalidation plus TTL is the usual pair.

Eviction

The shelf has a size. When it is full, something leaves.

- **LRU,** least recently used. Drop what nobody has touched in the longest time. A strong default.
- **LFU,** least frequently used. Drop what is used least often. Better when a few keys are steady favorites, and worse right after a key becomes popular, because it has a low count so far.
- **FIFO.** Drop the oldest resident, regardless of use.

Stampede

A popular key expires. A thousand requests miss at once and all query the database. That is a **thundering herd**, also called a cache stampede. Defenses:

- **Single flight.** The first miss loads the database. The others wait for that one load and share the result.
- **Lock on the key** with a short timeout, so only one loader runs.
- **Early refresh.** Before the TTL hits zero, one request refreshes the value while others still receive the old one.
- Serve a slightly stale value while a refresh runs. The reader is told, by your own rule, how old is still allowed.

What to cache, and what to leave

Cache the hot read that is expensive and reusable: TinyLink's code to long URL, a rendered feed page, a product page. Skip data that is unique per request and never reused. Skip data whose staleness would be a serious bug, such as a one-time password, unless the TTL is the feature. Remember that a cache is not the source of truth. You must be able to rebuild it.

A famous key that everyone wants is a **hot key**. One cache process can saturate on it. Defenses: replicate that key to several cache nodes and spread reads, or keep an extra copy in application memory for a second or two.

8. Names, doors, and distance

DNS

People type names. Machines use IP addresses. **DNS** answers “what address is `tiny.link`?” The answer can be cached at the laptop, at the company resolver, and at the DNS provider. A short DNS TTL lets you move traffic quickly. A long TTL makes the name survive a DNS outage and makes moves slow. The address you publish is often the load balancer, not a single application server.

Load balancers

A **layer 4** balancer forwards TCP connections. It is fast and unaware of HTTP. A **layer 7** balancer understands HTTP, so it can route `/images` to one pool and `/api` to another, and it can retry an idempotent GET against a different server.

Spreading policies: round robin, fewest active connections, or a hash of the client so the same client prefers the same server. Prefer-the-same-server is **sticky sessions**. Stickiness is a crutch for a server that kept session state in memory. Stateless servers do not need it. Health checks remove a dead server from the pool. The balancer itself must be redundant: two of them, with one address that can move.

CDN

A CDN caches bytes at the edge, near the user. The **origin** is your real closet. The first user in a city misses and pulls from the origin. Neighbors hit the edge. You control freshness with cache headers and TTLs. A **cache key** is whatever makes two responses different: the URL, and sometimes a header such as language. If you forget to include a distinguishing header, two users can receive each other’s personalized page. Personalized HTML usually stays uncached. Images, video segments, and public scripts cache well.

To purge, you delete by key or you change the URL (a new file name or query version). Changing the URL is the reliable kind of purge because old edges simply stop being asked.

9. Copies, consistency, and CAP

Replication

One notebook is a single point of failure and a single disk. Replication keeps copies.

- **Synchronous.** The leader waits until a follower has the write, then tells the client success. A crash loses less. The write is as slow as the follower.
- **Asynchronous.** The leader tells the client success, then the follower catches up. Writes stay fast. If the leader dies before the follower catches up, those last writes can vanish. The gap is **replication lag**.
- **Semi-synchronous.** The leader waits for one follower, and other followers catch up later.

Reads can go to followers to spread load. A follower may be a second behind, so a user who just wrote can read their old value. **Read-your-writes** is the fix for that feeling: after a user writes, that user's next reads go to the leader for a short time, or the read waits until the follower has passed that write's position.

Failover promotes a follower. Automatic failover needs a rule that avoids two leaders. Two leaders accepting writes is a **split brain**. The usual rule is a majority vote: a leader exists only if a majority of replicas agree it is the leader. A minority side stops accepting writes.

Quorums

With N copies, a write waits for W copies to acknowledge, and a read asks R copies. If R + W > N, the read set and the write set overlap, so the read sees at least one copy that has the latest write. A common choice is N = 3, R = 2, W = 2. Raising W favors fresher data and slower writes. Lowering R favors fast reads. This arithmetic is the heart of quorum systems. Leader-based databases get a similar result by sending the read to the leader.

CAP, then PACELC

A **partition** is a cut in the network: some machines cannot reach others. While the cut lasts, a system chooses between two behaviors. **Consistency** here means every successful read sees the latest successful write. **Availability** means every request receives a non-error answer. During a partition you emphasize one of them.

- A **CP** path refuses or waits when it cannot be sure. A bank transfer and a seat sale lean this way.
- An **AP** path answers with the best local copy and repairs later. A view counter and a presence dot lean this way.

PACELC adds the quiet hours. If there is a partition (P), you choose availability or consistency (A or C). Else (E), even when the network is fine, you choose latency or consistency (L or C). Waiting for two distant replicas is a consistency choice you pay for on every write, even on a healthy day.

TinyLink's redirect path answers from cache and from whatever database copy it is allowed to read. Click counts are allowed to lag. A custom-code insert demands the unique index on the leader, because two people must not both believe they own `priya`.

A ladder of "how fresh"

- **Strong / linearizable.** There is a single order of operations. A read sees the latest completed write.
- **Read-your-writes.** You see your own updates. Someone else might lag.

- **Monotonic reads.** You never go backward in time. Once you have seen version 5, you do not later see version 4.
- **Eventual.** Copies converge after updates stop. Before that, readers can disagree.

Name the rung the product needs. “We need strong consistency” for a whole product is rarely true. Payments need a high rung. Feeds work on a lower rung.

10. Sharding

Sharding splits rows across machines so each machine holds a fraction. You shard when one machine's disk, memory, or write rate is no longer enough. Replication copies the same rows for safety and read spread. Sharding divides rows. You usually do both: each shard has its own replicas.

How a row picks a home

Rule	How it works	Watch for
Range	Ids 1–1,000,000 on shard A, the next million on shard B	Simple, and the newest range takes all the inserts
Hash	hash(key) mod N	Even spread. Changing N moves almost every key
Consistent hash	Keys and servers sit on a ring. A key walks clockwise to the next server	Adding a server moves only the keys on the neighboring arc
Directory	A small lookup table says where the key lives	Flexible, and the directory itself must be fast and up

Consistent hashing in a little more detail. Picture a circle of positions. Each cache server is placed at several positions (virtual nodes), so one physical server owns many short arcs. A key is hashed onto the circle and stored on the next server clockwise. When a server leaves, its keys move to the next server on the ring. When a server joins, it takes the arc that now points at it. Virtual nodes keep one physical machine from owning a huge arc by luck. This is how shared caches and some databases place keys.

The shard key is the whole design

You can look up by the shard key with one hop. Any other lookup is a scatter: ask every shard, or maintain a second index. TinyLink shards by numeric id, and the code decodes back to the id, so the hot path stays one hop. If the hot path were "all links created this hour," a hash shard would force a scatter, and you would want a different key or a second store.

A **hot shard** receives far more than its share. A celebrity's messages, if you shard messages by user id and everyone writes on that user's shard, melt one machine. Fixes: split that user's data further, add a cache, or choose a key with more spread, such as conversation id.

Resharding moves rows to a new home. Do it as a copy, then a catch-up of new writes, then a short switch. Dual-write during the move, and check that both sides match before you trust the new home.

Cross-shard actions, such as a like from a user on shard A on a photo on shard B, do not get a single database transaction. You design them as two local writes plus a repair job, or you route them through a queue.

11. Queues and delivery promises

A queue lets the request path stay short. The server writes “please do this” onto a durable tray and answers the user. Workers pull notes and do the slow job: resize a photo, send mail, add a click, charge a card.

Promises

Promise	Meaning	Your job
At most once	The note is delivered zero or one time. A crash can lose it	Only for work you can afford to skip
At least once	The note is retried until a worker acknowledges it. A crash after the work and before the ack delivers it again	Make the worker idempotent
Effectively once	The combination of a durable tray and an idempotent worker produces one effect	Store the unique id of the note beside the effect, in the same transaction when you can

Most real systems give at least once. Design every consumer as if the same note will arrive twice. TinyLink’s click worker should add clicks in a way that a repeated note does not double-count, or you accept approximate counts and use at-least-once on purpose because a view counter may be approximate. Say which one you chose.

Ordering

A single queue with one worker preserves order. Several workers finish out of order. A log system such as Kafka restores order inside a **partition**: all messages with the same key, such as a conversation id, stick to one partition and stay ordered there. Different keys run in parallel. Order is per key, not global.

Parts of a log-shaped queue

- **Topic.** A named stream, such as `clicks`.
- **Partition.** An ordered shard of that stream.
- **Offset.** A position in a partition. The consumer records how far it got.
- **Consumer group.** A team of workers that split the partitions. Each partition is read by one member of the group at a time.
- **Retention.** The log keeps messages for a time or a size, whether or not they were read. New consumers can replay history. A classic task queue deletes on ack and does not replay a week later.

The outbox

A common bug: the database commits “order placed,” then the process crashes before it publishes “order placed” to the queue. The order exists and the mail never sends. The **outbox** writes the business row and an outbox row in the same database transaction. A publisher reads the outbox and publishes. If it crashes, it publishes again. Consumers are idempotent. The business change and the intent to publish commit together.

Dead letters and poison

A **poison message** makes every worker crash. After a few tries, move it to a **dead-letter queue** and alert a human. Retrying forever stops the line behind it. Retries should wait longer each time (backoff) and should add a little randomness (jitter) so a thousand workers do not retry on the same millisecond.

Backpressure means the producer slows down when the tray is deep. A queue that grows without limit hides a sick consumer until the disk fills. Cap the depth, or slow the producer, and alarm on age: “the oldest click has been waiting 2 minutes.”

12. Transactions, local and distributed

A **transaction** is a bundle of reads and writes that succeed or fail as one. Transferring money is two balance updates. Doing only one is a bug. Databases give transactions four properties, abbreviated ACID.

- **Atomicity.** All of the bundle commits, or none of it does.
- **Consistency.** The bundle moves the database from one valid state to another. Constraints and your rules hold.
- **Isolation.** Concurrent bundles do not see each other's half-finished work in the ways the isolation level forbids.
- **Durability.** After commit, the result survives a crash. The database writes a log to disk before it says yes.

Isolation levels

Level	What a transaction is protected from
Read uncommitted	Almost nothing. You can see another transaction's uncommitted write (a dirty read)
Read committed	Dirty reads. Each statement sees only committed data. Two statements in one transaction can still see different values of the same row
Repeatable read	A row you read stays stable for the rest of the transaction. A range can still gain new rows (a phantom) in some databases
Serializable	The result matches some one-by-one order of the transactions

Higher isolation costs more locking or more retries. Use the lowest level that keeps the product correct. Seat booking and balances need a high level or an explicit lock. A view counter needs neither.

Two transactions can deadlock: each holds a lock the other wants. The database aborts one. Your code retries the aborted one. Prevent the common case by locking rows in a consistent order, such as always locking the lower account id first.

Across services

A single SQL transaction stops at one database. Two services with two databases need a different plan.

Two-phase commit uses a coordinator. Phase one: every participant prepares and promises it can commit. Phase two: the coordinator tells them all to commit. If the coordinator dies between the phases, participants wait, holding locks. That blocking is why teams avoid two-phase commit across microservices, even though databases use related ideas internally.

A **saga** is a sequence of local transactions. Each step has a **compensating** action that undoes it if a later step fails. Place order, reserve stock, take payment. If payment fails, release stock and cancel the order. Compensations are forward actions, not time travel: a sent email is compensated by a second email, not by un-sending.

- **Orchestration.** One conductor calls the steps in order and runs compensations on failure. Easier to read.

- **Choreography.** Each service listens for events and does its step, emitting the next event. Less central coupling, and harder to see the whole story.

Sagas are atomic in spirit and visible in the middle. Other users can see “stock reserved, payment not taken” for a moment. Design those intermediate states as real product states.

13. Real-time connections

Some products need the server to speak first: a new chat message, a live score, a typing hint.

Technique	How it works	Fit
Short polling	The client asks “anything new?” on a timer	Simple. Wasteful when nothing is new. Fine for a rare check
Long polling	The client asks, and the server holds the request until something exists or a timeout hits. The client immediately asks again	Near-real-time without a special protocol
Server-Sent Events	One long HTTP response. The server pushes text events down. The client reconnects with “I last saw event 40”	One-direction updates: feeds, scores, notifications
WebSocket	A connection that stays open and carries messages both ways	Chat, games, collaborative editing

A stayed-open connection is state. The server that holds Priya’s socket is the only one that can push to her. Keep a **connection registry**: user id to gateway machine. When a message is born on any machine, it looks up the gateway and forwards. If Priya is offline, write the message to her inbox and send a phone push. Gateways must be many, and the registry must update on connect and disconnect.

Heartbeats find dead connections. Without them, a phone that lost its radio still looks online.

14. Feeds and fanout

A feed is “what should this person see, in order?” Two ways to build it.

Fanout on write. When Priya posts, the system writes the post id into the prebuilt feed of each follower. Opening the app is a cheap read of your own feed. A user with 10 million followers turns one post into 10 million writes. That user is a celebrity problem.

Fanout on read. The post is stored once. Opening the app looks up who you follow and merges their recent posts. Writes stay cheap. Reads get heavier as you follow more people, and the merge is on the hot path.

Hybrid. Ordinary accounts fan out on write. Celebrities fan out on read: at read time, merge your prebuilt feed with the latest posts from celebrities you follow. This is the usual answer for a large social product.

Ranked feeds add a scorer: recency, affinity, and a few content features. Scoring can run on write (store the score) or on read (rescore the candidates). Candidate generation plus ranking is the shape: pick a few hundred possible posts cheaply, then rank those, then take the top twenty. You do not rank the entire history on every open.

Cursors keep paging stable. Store “the last post id the client rendered” and ask for older ones.

15. More than one city

One data center is one building. A fire, a cut cable, or a bad deploy takes it. Multi-region design is how you keep going.

- **Active-passive.** One region serves. Another holds replicas and is ready to take over. Failover takes minutes and may lose the last asynchronous writes. Simpler to reason about.
- **Active-active.** Two regions both serve users, usually the users nearest them. Writes in both places can conflict. You need a rule: route a given user's writes to a home region, or use conflict-free data, or accept last-writer-wins for data where that is harmless.

Home region is the clean rule for accounts and payments: Priya's rows live in one region, and her writes go there even if she travels. Reads of her own data go there too, or to a local replica that is allowed to be slightly behind. Global public data, such as a viral video file, replicates everywhere through the CDN and object storage.

Latency across an ocean is tens to a couple of hundred milliseconds. A design that does five cross-ocean database calls on one tap will feel broken. Keep the hot path inside one region.

16. Staying up

Failures are ordinary. A design names them.

- **Timeout.** Stop waiting. Every remote call has one.
- **Retry.** Try again for errors that might vanish: a dropped packet, a 503, a timeout. Do not retry a 400. Retry only idempotent operations, or carry an idempotency key. Use exponential backoff and jitter. Cap the number of tries.
- **Circuit breaker.** When a neighbor is failing most calls, stop calling for a short cool-down. A few trial calls check whether it has recovered. This keeps your servers from piling up threads on a sick neighbor.
- **Bulkhead.** Separate pools of threads or connections for separate neighbors. A stuck payment call must not consume the threads that serve the homepage.
- **Rate limit.** Cap how fast one caller, one IP, or one API key may knock. Algorithms are in chapter 18.
- **Load shedding.** When you are overloaded, refuse new work quickly with 503 so the work you did accept can finish. A server that accepts everything and then does it slowly fails everyone.
- **Graceful degradation.** If recommendations are down, show the plain feed. The product continues in a smaller form.
- **Idempotency keys.** The client sends a unique token. The server stores the token and the response. A retry returns the stored response.

Define a **service level objective** in plain numbers: “99.9% of opens complete under 100 ms in a month.” An **SLI** is the measurement (the percentage that met the bar). An **SLA** is the promise to a customer, often with a consequence. Design the system to the objective, and alarm when the measurement approaches it.

17. Security

Security in system design is a set of boxes and checks, not a coat of paint at the end.

- **TLS.** Encrypts data on the network so a stranger on the path cannot read or alter it. Terminate TLS at the load balancer or at the service, and be deliberate about which hop is encrypted inside the data center too.
- **Passwords.** Store a slow hash made for passwords (bcrypt, scrypt, or Argon2) with a unique salt. A hash is one-way. Encryption, which can be reversed with a key, is the wrong tool for passwords. The database never needs the original password again; it only needs to check a new hash against the stored hash.
- **Sessions and tokens.** After login, the client holds a token. A random session id stored on the server can be revoked by deleting the row. A **JWT** is a signed bundle of claims (user id, expiry). Anyone can read it. Only the holder of the signing key can mint a valid one. The server checks the signature and the expiry. Revoking a JWT before expiry needs an extra denylist or very short expiry plus a refresh token.
- **OAuth.** The user signs in with a specialist (the identity provider). Your app receives a token and never sees the password. Your app still checks authorization itself.
- **Authorization on the server.** The client can hide a button. The server still checks “this user owns this link” on every call. Hiding a button is a convenience. The check is the control.
- **Least privilege.** The photo service’s database user cannot read the payments tables. A leaked credential then reaches a smaller blast radius.
- **Encryption at rest.** Disks and backups are encrypted so a stolen drive is not a stolen database. Keys live in a key service, separate from the data.
- **Input rules.** Validate type, length, and allowed characters. Parameterized queries keep user text from becoming SQL. Object keys are generated by you, so a user cannot name a key that overwrites someone else’s file.
- **Rate limits on login and create.** They cut password guessing and spam.
- **Secrets.** Keys live in a secret store or environment, never in the repository and never in the client application.

18. Seeing a running system

If you cannot see it, you cannot keep the objective.

- **Logs.** A line per meaningful event, with a timestamp, a service name, and a **request id** that every hop copies. Structured fields (user id, code, latency) beat a paragraph of prose when you search later.
- **Metrics.** Numbers over time. For a request path, watch rate, errors, and duration. For a machine, watch CPU, memory, disk, and saturation of the pool. For a queue, watch depth and age of the oldest message.
- **Traces.** One request's spans across services: 12 ms in the gateway, 4 ms in the cache, 40 ms in the database. Traces answer "where did the time go?" for a single slow tap.
- **Alerts** page a human on symptoms that need action: error rate over the objective, queue age over the budget, disk predicted to fill. A chart you look at in the morning is a dashboard. An alert is a page. Page on few things, or people will sleep through them.

19. One program or many

A **monolith** is one deployable program with modules inside it. A **microservice** is a program that owns one business capability and its data, and talks to others over the network.

Start with one program and clear modules while a small team is learning the product. Split a piece out when it has its own scaling need, its own release clock, or its own strict data boundary. TinyLink's workers can be a module in the same program. A payments system beside a website is a natural split because the security boundary and the consistency needs differ.

Each service owns its tables. Another service asks through the API or listens to events. Reaching into another service's tables couples you to its private shape and breaks the moment that shape changes.

An **API gateway** is the front door for external clients: TLS, authentication, rate limits, and routing to services. Internal services may talk directly for speed, still with timeouts.

Service discovery is how a caller finds a healthy instance. A registry tracks "payment service, these three addresses." The load balancer or a client library consults it.

gRPC is a binary request protocol with a strict contract, often used between services. HTTP JSON is easier for browsers and public APIs. Both are just menus of calls. The design questions in this book do not change when the bytes on the wire change.

20. Tools you will reach for

Unique ids

- **Database sequence.** Simple, ordered, and a single allocator. Fine at TinyLink's 40 writes per second.
- **Blocks.** Each server leases a range. Fewer round trips. Holes appear when a server dies. Order is roughly increasing.
- **Snowflake-style 64-bit ids.** A timestamp, a machine number, and a per-machine counter. Ids are unique and roughly time-ordered without a central allocator. Clocks must stay close enough that you do not mint the same timestamp-and-counter pair; the usual design uses a sequence within the same millisecond so a clock wobble of one millisecond is handled by waiting or by the counter.
- **Random UUIDs.** Unique with extremely high probability and no coordination. They scatter in B-tree indexes, which makes inserts more random. Fine as public tokens. Less fine as the only primary key of a huge high-insert table, unless the database is built for random keys.

Do not use raw auto-increment ids in a public URL if guessing the next id would expose someone else's data. TinyLink's codes are public handles on purpose; private resources should use unguessable tokens plus an authorization check.

Hashing

A hash function maps bytes to a fixed-size number. Tiny changes in the input swing the output. Hashes place cache keys, shard keys, and checksums. A cryptographic hash (SHA-256 and its cousins) is what you want when an attacker must not be able to craft a collision. A fast non-cryptographic hash is what you want for a hash table. Password storage uses a slow password hash, which is a different tool from either of those.

Bloom filters

A Bloom filter answers "have I seen this item?" with two results: **definitely no**, or **maybe yes**. It is a row of bits. Inserting an item sets k bit positions chosen by k hashes. Asking an item checks those k bits. If any bit is off, the item was never inserted. If all are on, it was probably inserted, or other items happened to set the same bits.

Use one in front of a huge store to skip useless lookups: a crawler asks "have we fetched this URL?" and a definite no avoids the database. A maybe-yes still checks the database. You cannot delete an item from the basic filter, because clearing a bit might belong to another item. Size the bit array from the number of items and the false-yes rate you allow.

Search

A database `LIKE '%cat%'` scan does not survive a large corpus. A **search index** inverts the text. The **inverted index** maps a term to the list of documents that contain it: `cat -> doc 4, doc 9, doc 20`. A query intersects lists. Ranking then orders the hits using signals such as how rare the term is and where it sits in the document.

The index is derived. The source of truth stays in your database or object store. An indexer consumes a stream of creates and updates and applies them to the index. Search being a few seconds behind the write

is normal and should be stated as a product choice. Tokenization, language, and typos are the hard product edges: decide whether “Cats” matches “cat,” and whether a typo returns neighbors.

Autocomplete is a prefix problem. A trie, or a prefix index, maps `ca` to the ten most common completions. Those top-ten lists can be precomputed from recent query counts so the keystroke path is one cache read. Update the counts asynchronously. Personal suggestions are a second, smaller list merged at read time.

Rate limit algorithms

Algorithm	Behavior
Fixed window	A counter per clock minute. Simple. At the boundary, a caller can send the full allowance twice in two seconds, once at 12:00:59 and once at 12:01:00
Sliding window log	Store each timestamp and count those inside the last minute. Accurate. Uses more memory
Sliding window counter	Keep the previous window’s count and the current one. Estimate the last minute as a weighted mix. Small memory, small error
Token bucket	Tokens drip in at a steady rate, up to a bucket size. Each request takes a token. The bucket size is the allowed burst. Empty bucket means refuse
Leaky bucket	Requests join a queue that drains at a fixed rate. Bursts are smoothed. Excess spills (is refused) when the queue is full

Store the counters in a shared cache so every application server enforces the same limit. The key is the caller identity plus the route. Return 429 and a hint about when to retry.

21. Worked high-level designs

Each design follows the same spine and only spends words on what is new. Numbers are assumptions you should recompute in practice.

21.1 Rate limiter

Job. Allow N actions per identity per window. Refuse the rest with 429.

Needs. Correct enough under several application servers. Fast, because it sits on every request. A small burst may be allowed. Different routes have different allowances.

Size. At 8,000 requests per second, the limiter runs 8,000 checks a second. Each check is a cache increment, not a SQL transaction.

Design. A shared cache holds a token bucket per key. The key is `userId:route` or `ip:route`. The value is the token count and the last refill timestamp. Refill is computed from elapsed time, so you do not need a timer per key: `tokens = min(capacity, tokens + elapsed × rate)`. Then subtract one if a token remains.

Do the read-modify-write inside one atomic cache script so two servers cannot both spend the last token. Put the limiter in the gateway and also in the service that must protect itself if the gateway is bypassed.

Fails. If the cache is down, decide in advance: fail open (allow traffic) or fail closed (refuse). Login limits fail closed. A homepage may fail open. State the choice.

21.2 Unique id service

Job. Mint ids that are unique, roughly time-ordered, and available when any one application server is up.

Design. Snowflake-style 64-bit numbers. Reserve 1 bit, 41 bits of milliseconds since a custom epoch (about 69 years), 10 bits of machine id (1024 machines), 12 bits of sequence (4096 ids per machine per millisecond). A machine that would exceed 4096 in the same millisecond waits until the next millisecond. Machine ids come from a small coordinator or from the orchestration system, and they are unique while the process lives.

Clock going backward is the bug. If the new timestamp is older than the last one used, wait until the clock catches the last timestamp, or refuse to mint and alarm. Do not reuse sequence numbers in the past.

21.3 Chat

Functional. One-to-one and group messages, history, online presence, delivered and read marks, and a push when the person is offline.

Non-functional. Delivery under a second when both are online. History durable. Order stable inside one conversation. A group may have hundreds of members; a broadcast channel is a different product.

Size assumption. 50 million daily senders, 20 messages each, is 1 billion messages a day, about 12,000 writes per second average, perhaps 30,000 at peak. Each message is about 200 bytes of metadata plus text. A year is on the order of tens of terabytes, sharded.

API and connection. WebSocket for send and for live receive. `GET /conversations/{id}/messages?before=cursor` for history. `POST` is a fallback when the socket is down.

Data. Conversations, members, and messages keyed by `(conversation_id, message_id)`. Shard by conversation id so one history is one shard and stays ordered. A per-conversation increasing id, or a Snowflake id plus a gap-filling rule, gives order. Inbox pointer: for each user, the list of conversation ids and the latest message id, so the chat list is one read.

Path. Client sends on its socket to Gateway A. Gateway A writes the message to the store, then looks up each member's gateway in the connection registry and forwards. If a member has no gateway, it writes a push-notification job. The sender receives an ack with the message id. Recipients send delivered and read events, which update small rows and fan out the same way.

Presence is a TTL key in the cache, refreshed by heartbeat. It is allowed to be wrong for seconds.

Hard parts. Gateway fanout for a large group: write once, then push to N gateways, not N database copies of the body. Deduplicate retries with a client message token. Do not let one huge group stall a worker: cap fanout parallelism. Media goes to object storage; the message holds the key.

21.4 Photo feed

Functional. Post a photo and a caption, follow people, open a home feed, like, comment.

Non-functional. Feed open is fast. Posting returns when the photo is safe to acknowledge, not when every follower's feed is updated. A celebrity can have millions of followers.

Size assumption. 10 million daily posters, one photo each. 100 million daily feed opens. Photos average 2 MB. That is about 20 TB of new images a day before you count replicas and thumbnails. Metadata is the smaller SQL problem. Bytes are the object-storage and CDN problem.

Path, post. Client uploads the file to object storage, directly with a signed upload URL so the bytes do not pass through your application server. The server writes the photo row, enqueues thumbnail and abuse-scan jobs, and enqueues fanout. The client sees the post as soon as the row exists. Followers see it when fanout or read-merge reaches them.

Path, feed. Hybrid fanout from chapter 14. Cache the first page of each user's prebuilt feed. Likes can be counted in a cache and flushed, with the exact count allowed to lag, or stored as rows if you must list who liked.

Data. Users, follows (follower_id, followee_id), photos (id, owner, object key, time), feed entries (user_id, photo_id, time). Shard follows and feed entries by the reader's user id, because the hot read is "my feed." Shard photos by photo id. The celebrity is excluded from write-fanout and merged at read.

Hard parts. Upload resumes for bad networks: object storage multipart upload. Thumbnail so the feed never downloads full-size images. Authorization on every photo URL: private photos use signed URLs that expire, public photos sit on the CDN.

21.5 Notifications

Job. Tell a user that something happened, on the channels they allow: in-app, mobile push, email, maybe SMS.

Path. A service emits an event: "photo 9 was liked by user 3." A notification service looks up subscribers and preferences, writes an in-app row, and enqueues one job per external channel. Channel workers talk to providers. A preference of "no email" drops that job. A rate cap stops one viral photo from emailing you fifty times in a minute: collapse them into "Priya and 49 others liked your photo."

Data. Notification rows for the in-app inbox, keyed by user id. Delivery log with provider ids so retries are idempotent. Preferences per user per event type per channel.

Hard parts. Provider outages go to retries and a dead-letter queue. Idempotency key is the event id plus channel plus user. Quiet hours are a product rule in the worker, not a special database.

21.6 Video streaming

Functional. Upload a video, play it, pause, and switch quality as the network changes.

Non-functional. Playback starts quickly. Upload may take minutes. Many viewers watch a few popular videos.

Upload. Multipart upload to object storage. A row is created in a "processing" state. A queue runs transcoding into several bitrates and splits each into short segments (a few seconds). A manifest file lists the segments. When the ladder of qualities is ready, the row becomes "playable."

Playback. The client fetches the manifest from your API or from the CDN, then fetches segments from the CDN. The client picks a bitrate that fits the current bandwidth. Your origin sees a segment only on a CDN miss.

Data. Video metadata in SQL. Segments in object storage. View counts via a queue, like TinyLink clicks. Recommendations are an offline or nearline job writing "related ids" back to a store the play page can read quickly.

Hard parts. Popular videos are a CDN problem, not an application-server problem. Live video is the same shape with smaller segment duration and a manifest that grows. Protect upload with quotas so one client cannot fill the bucket.

21.7 File sync

Job. A folder on a laptop stays in agreement with a folder in the cloud and with other devices.

Design. Split files into chunks of a few megabytes. Hash each chunk. Upload only chunks the server does not have, which also deduplicates identical chunks across users if you are careful about privacy and encryption. Metadata is a tree: folders, file names, and an ordered list of chunk hashes. Each save creates a new version rather than overwriting the only copy, so you can undo.

Sync. The client sends a local change list. The server applies it when it does not conflict. Two devices editing the same file while offline create two versions; the product shows a conflict and keeps both. A cursor per device says "I have seen everything up to change 500."

Hard parts. Chunk hashes must be strong. Encryption: if the server stores only ciphertext, deduplication across users gets much harder, and you should say which goal wins. Sharing a folder is an authorization list on the metadata, plus a way to revoke.

21.8 Search and autocomplete

Search. Documents live in the source database. An indexer tails the change stream and updates an inverted index. Queries hit a search service that fans out to index shards and merges the top results. Cache frequent queries. Ranking features that change slowly can be stored beside each post; features that change every second, such as "liked one second ago," are applied only to the top few candidates.

Autocomplete. On each keystroke, read a precomputed top-10 for that prefix from cache. Build those lists from aggregated query logs in a job that runs every few minutes. Filter with a blocklist before the list

is published. Personal history is merged from a small per-user cache. The latency budget is tight, often under 50 ms, so this path is cache only. A miss returns an empty list or a shorter prefix's list, and a job fills the missing prefix later.

21.9 Web crawler

Job. Given some seed URLs, fetch pages, extract links, and repeat, without melting any host and without fetching the same URL forever.

Pieces. A URL frontier (a queue, ideally prioritized and partitioned by host). Fetchers. A parser that extracts links and content. A dedup store. A politeness rule: at most one fetch at a time to a host, with a delay. A Bloom filter in front of the dedup store answers "definitely new" quickly; "maybe seen" checks the store.

Hard parts. The frontier is the product. If you use one queue, a huge site crowds out the rest. Partition by host so politeness and fairness are natural. Respect a site's crawl rules as a product and legal requirement. Store content in object storage and metadata in a table. Retries need a cap because some URLs never succeed.

21.10 Payments

Job. Take a payment for an order exactly once, know the state afterward, and be able to reconcile with the outside provider.

Design. Your service is not the bank. It talks to a payment provider. The order service creates a payment row in state `pending` with an idempotency key. It calls the provider with that same key. The provider's webhook, and a background poll, move the row to `succeeded` or `failed`. The webhook and the poll can both arrive; the state machine ignores a repeated success. Money movement is recorded as a **ledger**: append-only entries of debit and credit, not a single balance that you edit in place. The balance is the sum of entries, or a cached sum you can rebuild.

State machine. `pending → succeeded`, `pending → failed`, `succeeded → refund_pending → refunded`. There is no path from failed to succeeded without a new payment attempt and a new idempotency key.

Hard parts. The provider said yes and your process died before you stored it. The poll or the webhook heals this; both are required because webhooks get lost. Never trust the browser's "payment worked" as the source of truth. Reconciliation compares your ledger to the provider's report every day and alarms on mismatches.

21.11 Metrics and monitoring

Job. Ingest numeric samples from many services and answer "what was the error rate over the last five minutes?"

Design. Each service emits samples: name, tags, timestamp, value. A collector aggregates locally (sum, count, histogram buckets) so you do not ship every request as its own row. Aggregates flow to a time-series store, partitioned by metric name and time bucket. Queries read pre-aggregated buckets, not raw events. Alert rules run on the same store on a schedule.

Hard parts. Cardinality. A tag whose value is a user id creates millions of series and melts the store. Tags are low-cardinality labels such as service, route, and status. Histograms beat averages, because an average hides a slow tail. Keep raw logs somewhere else; this system stores numbers.

21.12 News ticker / pub-sub

Job. Publishers emit topics. Subscribers receive the topics they joined.

Design. A topic is a log partition set. Publishers append. Subscriber groups track offsets. For live UI subscribers, a gateway holds the socket and reads the log. Retention keeps history for replay. This is the queue chapter specialized to many independent readers. The new point is fanout to readers: a message stays in the log once, and each group has its own offset. You do not copy the payload per reader inside the log.

What these designs share

- A source of truth, and derived copies (cache, search, feed, CDN) you can rebuild.
- A short path for the user and a queue for the slow part.
- An idempotency key wherever a retry could double-charge, double-post, or double-send.
- A shard key taken from the hot lookup.
- A stated freshness rule: strong for money and unique names, lagged for counts, presence, and search.

22. The LLD method and object rules

Low-level design starts after the city map exists, or beside it when the problem is mostly rules inside one program. The reader of your LLD should be able to write the code.

Steps

1. Use cases, written as actor plus action.
2. Nouns become classes. Messages become small data objects.
3. Relationships: who calls whom, and who holds a list of whom.
4. Methods on the service that match the use cases. Walk the hottest use case line by line.
5. Interfaces at the edges: database, cache, clock, random, payment provider. Your rules stay testable without those neighbors.
6. The table, the unique keys, and the update that is safe under two writers.
7. Edges: empty input, duplicates, expiry, crashes, and two actions at once.

Objects

A **class** is a blueprint. An **object** is one built instance. A **field** is data the object holds. A **method** is an action it can do.

Encapsulation. Fields stay private. Callers use methods. The class can change how it stores data without changing every caller.

Abstraction. Callers see a small promise, such as `findByCode`, and do not see SQL.

Inheritance. A class is a special kind of another class and reuses its code. Use it when the sentence "is a kind of" will stay true forever. A `Bus` is a kind of `Vehicle`.

Polymorphism. A caller holds the general type and gets the specific behavior. A list of `Vehicle` calls `spotsNeeded()`, and a bus returns 5 while a motorcycle returns 1.

Composition. A class holds other objects and asks them to help. `LinkService` holds a `Codec`. Prefer composition as the default way to reuse behavior. Inheritance is for a true "is a kind of" relationship.

An **interface** is a plug shape with no stored data: a list of methods. An **abstract class** is a partial blueprint: some methods implemented, some left open, and it may hold fields. Use an interface when many unrelated classes should fit the same plug. Use an abstract class when the shared code is real and the children are the same family.

SOLID

- **Single responsibility.** One reason to change. A codec that also opens SQL connections changes when the encoding changes and when the database changes. Split them.
- **Open for extension, closed for modification.** A new fee rule is a new class, not an edit inside `leave()`.
- **Liskov substitution.** Anything passed in as the interface must keep the promise. If `findByCode` says "return null when missing," every implementation does that. A surprise exception breaks callers.
- **Interface segregation.** Small plugs. A class that only inserts links does not implement a 30-method administration interface.

- **Dependency inversion.** The service depends on the plug, not on the SQL class. The SQL class is passed in at construction. Tests pass a fake.

A readable shape for request code

```
Controller  - HTTP, status codes, auth checks at the door
Service     - rules and decisions
Repository  - load and save
Entity      - the fact, with small helpers that never touch the network
Clock, IdSource, Cache, Queue - plugs
```

Validation of shape (missing field, too long) can live at the controller. Validation of meaning (this seat is taken, this code is expired) lives in the service, because it is a rule even when the caller is not HTTP.

UML you actually draw

Class diagram. A box per class. A line with a diamond means “owns a collection of.” An arrow means “calls” or “holds.” A dashed arrow to an interface means “fits this plug.” You do not need every field on the diagram. Put the fields that explain the design.

Sequence diagram. Lifelines across the top, time downward, arrows for calls. Draw one for the hot use case and one for the failure you are proud of handling.

State diagram. Bubbles for states, arrows for events. Draw one whenever a thing has a lifecycle: payment, seat hold, elevator, vending machine. If you cannot name the states, you do not understand the rules yet.

23. Patterns

A pattern is a named shape. Use it when the problem matches. Naming it is how you talk to other programmers. The shape matters more than the name.

Pattern	Shape	Use when
Strategy	Several classes share a method. The caller holds one and can be given another	Fee rules, id generators, ranking, eviction policies
State	An object's behavior changes by swapping a state object. Each state decides the next	Elevator, vending machine, payment, TCP-like flows
Observer	A subject keeps listeners and tells them when something happens	In-process events: "seat booked, tell the display." Across processes, use a queue instead
Factory	One method builds the right class so callers do not construct variants themselves	Build a Vehicle from the string "bus"
Builder	Step-by-step construction of an object with many optional fields, ending in `build()` which checks the required ones	A search query, a notification with many channels
Adapter	A wrapper that makes a foreign class fit your plug	Two payment providers behind one `Charger` interface
Facade	One simple method in front of a cluster of classes	`Checkout.place(order)` hides inventory, payment, and mail
Decorator	A wrapper with the same interface that adds behavior and then calls the inner object	Logging, timing, or compression around a repository
Proxy	A stand-in that controls access to the real object	A lazy loader, a permission check, a local stub for a remote service
Composite	Leaves and branches share an interface. A branch holds children	File system folders, UI trees, organization charts
Command	An action is an object with `execute()` and maybe `undo()`	Keybindings, job queues in memory, editor operations
Chain of responsibility	Handlers in a line. Each handles the request or passes it on	HTTP middleware: auth, then rate limit, then the handler
Template method	A base class fixes the steps and calls overridable hooks	A game loop, an import pipeline with a custom parse step
Iterator	A way to walk a collection without showing its insides	Paging through seats or files
Repository	A collection-like plug for loading and saving entities	Any LLD that touches a database
Dependency injection	Pass helpers into the constructor	Always, for anything you want to fake in a test
Singleton	One instance in the process	Rarely. A stateless codec can be one shared object. A hidden global database connection makes tests lie. Prefer passing the object in

24. Concurrency inside one process

Two threads of the same program can run your method at the same time. The bugs are real even when you have one server.

A **race** is two threads reading a value, both adding one, and both writing back. One update disappears. A view counter implemented as read-modify-write in application memory has this bug. Fixes: an atomic increment, a database expression `set count = count + 1`, or a lock around the whole read-modify-write.

A **lock** (mutex) lets one thread into a block at a time. Keep the block short. Do not call the network while holding a lock that other requests need.

A **deadlock** is two threads each holding a lock the other wants. Prevent it by acquiring locks in a global order: always lock the smaller id first. If you already hold locks and need a third, you probably want a redesign.

A **thread pool** is a fixed set of workers pulling tasks. Unbounded thread creation under load exhausts memory. Queue in front of the pool, and when the queue is full, reject.

Immutable objects never change after construction. They need no lock to be shared for reading. A new version is a new object.

In an LLD interview or a design doc, say which data is touched by two threads and which of these tools you use. “The seat map is guarded by one lock per show, and the lock is held only around the in-memory check and the database conditional update” is a complete answer. A conditional update, `update seats set owner = ? where id = ? and owner is null`, lets the database be the lock. Check the affected-row count. Zero means someone else took it.

25. Memory structures

Pick the structure from the operation that must be fast.

Structure	Fast operation	Designs that need it
Dynamic array	Index by position, append	A board, a list of segments
Linked list	Insert and delete when you already hold the node	The order part of an LRU cache
Hash map	Get, put, delete by key, average constant time	Lookups by id, the index part of an LRU cache, a connection registry
Hash set	Membership	“Is this ticket already seen?”
Heap / priority queue	Take the smallest or largest	Elevator requests by floor distance, top-K, scheduled jobs by time
Balanced tree / ordered map	Range and nearest key, sorted order	“Next free seat number,” calendars
Trie	Prefix lookup	Autocomplete, dictionaries
Graph	Neighbors, paths	Follows, maps, dependency trees
Deque	Push and pop at both ends	Sliding windows, BFS

State the cost you care about. “Get and put are both O(1)” is the whole reason an LRU cache is a hash map plus a doubly linked list, and not a list you scan.

LRU cache, because it is both a structure and a design

Goal: `get` and `put` in constant time, and when capacity is exceeded, drop the least recently used key.

A hash map gives the node for a key. A doubly linked list gives order: the head is most recently used, the tail is least. `get` moves the node to the head. `put` of a new key adds at the head and, if over capacity, deletes the tail from the list and the map. `put` of an existing key updates the value and moves the node to the head. The map and the list always contain the same keys. One lock around both updates keeps them agreeing, or the structure is used by a single thread inside a shard.

26. Testing a design

A design you cannot test is unfinished. Write tests against the service, with fakes for the plugs.

- **Repository fake** is a hash map. It enforces the same unique keys the database would, so a test can prove the 409 path.
- **Clock fake** returns a time you set, so expiry tests do not sleep.
- **Queue fake** records published notes.
- **Cache fake** is a map that can be cleared to simulate a flush.

Cases worth writing before you trust the service:

- The happy path of each use case.
- Invalid input.
- Not found, and found-but-expired.
- The same request token twice.
- Two creates with the same custom code.
- A cache hit, a cache miss, and a cache that throws. The service should still read the database when the cache throws, if that was your HLD choice.
- A conditional update that affects zero rows.

Concurrency tests start two threads on the last seat or the last token and assert that exactly one wins. Flaky tests that sleep and hope are weak. A deterministic interleaving, or a database constraint you can observe, is strong.

27. Worked low-level designs

These are class-level designs. Each one names the states, the plugs, and the race that matters. Pseudocode is a sketch of the rules, not a library you should paste into production without the checks your environment needs.

27.1 TinyLink inside the server

```
Link
  id, code, longUrl, createdAt, expiresAt, clickCount, requestToken

CreateLinkCommand
  longUrl, customCode or empty, expiresAt or empty, requestToken

LinkService
  create(command) -> Result
  open(code, now) -> Result
  stats(code) -> Result

plugs: LinkRepository, LinkCache, IdSource, Base62Codec, ClickQueue, Clock
```

`create`: reject a bad URL in the controller. If `requestToken` already exists, return the stored link. If `customCode` is set, insert it and translate a unique-key failure into "taken." Otherwise `id = ids.next()`, `code = codec.encode(id)`, insert. Cache after commit.

`open`: cache get. On miss, repository get, then cache put. Missing means not found. `expiresAt` before `now` means gone. Otherwise publish a click and return the long URL. If the cache throws, read the repository anyway.

The repository's insert is the uniqueness boundary. The service does not check-then-insert without handling the loser of the race. Two custom-code requests can both see "free" and both insert; the database lets one commit.

27.2 Parking lot

Rules. Floors contain rows contain spots. A motorcycle fits a motorcycle spot or larger. A car fits a car spot or a bus-sized run. A bus takes five consecutive free car spots in one row. A ticket records the vehicle, the spots, and the entry time. Leaving frees the spots and charges a fee.

```
enum SpotSize { MOTORCYCLE, CAR }

Spot
  id, size, free

Vehicle
  plate, spotsNeeded(), preferredSize()
  Motorcycle 1, Car 1, Bus 5

Ticket
  id, plate, spotIds, entryTime

ParkingLot
  park(vehicle, now) -> Ticket or Full
  leave(ticketId, now) -> Fee

FeeCalculator  (strategy)
  fee(entry, exit) -> money
```

`park` walks rows and calls `row.allocate(vehicle)`. For a bus, a row scans for five adjacent free car spots and marks them taken together. The walk and the mark happen under a lock per lot, or as a database transaction that re-reads the spots. Then a ticket is saved. If the process crashes after marking spots and before the ticket, a repair job frees spots that have been held with no ticket past a few seconds. Cleaner approach: insert the ticket and the spot updates in one transaction.

`leave` loads the ticket, marks those spots free, asks the fee strategy, and stores the exit time. Leaving twice returns the same fee and does not free spots twice. That is the idempotent path.

A new fee rule is a new `FeeCalculator`. The lot does not grow a chain of if-statements.

27.3 Logger

Rules. Callers log a level and a message. The logger writes to one or more appenders: console, file, or a queue. Messages below the configured level are dropped. A change to the level takes effect without restarting callers.

```
enum Level { DEBUG, INFO, WARN, ERROR }

LogEvent
  time, level, loggerName, message, context map

Appender
  append(event)

Logger
  name, minLevel, appenders
  log(level, message)
    if level < minLevel: return
    event = build
    for appender in appenders: appender.append(event)

LoggerRegistry
  get(name) -> Logger
```

The registry is the one shared object, injected where you can, looked up where legacy code forces it. Appenders that do I/O belong behind an async buffer with a bounded queue. When the queue is full, drop debug events and count the drops, or block callers; pick one and document it. Formatting is a strategy so

JSON and plain text are swappable. Do not let a failing appender throw into the caller's business path: catch, count, and continue with the other appenders.

27.4 Elevator

Rules. One car for the core design. Floors call up or down. The car has buttons for destination floors. The car serves requests in its current direction, then reverses. This is the LOOK algorithm: go as far as the furthest request in this direction, then turn around. It does not move toward a call it will pass only after reversing, until the current direction is exhausted.

```
enum Direction { UP, DOWN, IDLE }

Request
  floor, direction or destination

Elevator
  currentFloor, direction, upStops set, downStops set
  call(floor, direction)
  chooseFloor(floor)
  step()  // move one floor or go idle

Scheduler
  // for several cars: pick the car whose extra distance is smallest
  assign(call)
```

`step` is the heartbeat. If there is a stop at the current floor in this direction, open doors and clear that stop. Else if stops remain ahead, move one floor. Else if stops exist the other way, flip direction. Else go idle. Internal destination buttons and hall calls both become stops. A hall call upward is a stop only when the car is traveling up or is idle and chooses up.

Several cars: the scheduler assigns a new hall call to the car that can reach it with the least added travel, given that car's current direction. Once assigned, the call lives on that car. State pattern fits if you prefer classes `Idle`, `MovingUp`, `MovingDown`, `DoorsOpen`. The `step` description above is the same machine written as data.

27.5 Movie tickets

Rules. A show has a screen and a start time and a set of seats. A user holds seats for a few minutes while paying, then confirms. Another user cannot take held or sold seats. After the hold expires, the seats return.

```
Seat
  id, showId, label

Booking
  id, showId, seatIds, userId, state, holdExpiresAt
  states: HELD, CONFIRMED, CANCELLED, EXPIRED

BookingService
  hold(showId, seatIds, userId, now) -> Booking
  confirm(bookingId, paymentToken, now)
  expireHolds(now)
```

The race is two people holding the last seat. Do not check availability in memory and then insert. Update rows: `update seats set hold_id = ? where show_id = ? and seat_id in (...) and hold_id is null and sold = false`. If the number of updated rows is smaller than the number requested, roll the transaction

back and return “unavailable.” Confirming sets `sold = true` only if this booking still holds them and payment succeeds. Payment uses the payments design: idempotency key equals booking id.

`expireHolds` is a worker. It clears holds whose expiry has passed and whose state is still HELD. Confirm and expire can race; the conditional update lets only one win. If payment succeeds and expiry wins first, the payment saga compensates with a refund. Shorten that window by expiring only holds older than the limit and by confirming inside a transaction that rechecks the hold.

27.6 Splitwise

Rules. Users share expenses in a group. Each expense has a payer and a list of shares that sum to the total. The product shows who owes whom. Settling records a payment that reduces debt.

```
Expense
  id, groupId, payerId, amount, time
Share
  expenseId, userId, amount
Settlement
  fromUser, toUser, amount, time

Ledger
  // derived: for each pair, net balance
  balances(groupId) -> list of (from, to, amount)
  addExpense(expense, shares)
  settle(from, to, amount)
```

Store expenses and shares as the source of truth, append-only. A balance is the sum of shares minus what has been settled. You may cache balances and rebuild them. Simplify debts for display with a greedy pass: people with positive net and people with negative net, match the smaller amount, repeat. Say that the simplification is a display of the same nets, not a deletion of history. Shares must sum to the expense. Reject the expense otherwise. Edits are new adjusting entries, or a reversal plus a new expense, so history stays explainable.

27.7 ATM

Rules. Read a card, check a PIN, show accounts, dispense cash, or take a deposit. The bank is an external service. The ATM can be cancelled at each step. Cash inventory is local and must not go negative.

```
states: Idle, CardInserted, Authenticated, Dispensing, OutOfService

Atm
  state
  insertCard(card)
  enterPin(pin)      // calls Bank.authenticate
  withdraw(account, amount)
  cancel()

CashDrawer
  notes: map of denomination -> count
  canDispense(amount)
  plan(amount) -> notes or Cannot
  commit(plan)

Bank (plug)
  authenticate(card, pin)
  debit(account, amount, idempotencyKey)
```

`withdraw`: plan the notes locally without dispensing. Call `bank.debit` with an idempotency key. On success, commit the notes out of the drawer and open the slot. On failure, dispense nothing. If the debit succeeded and the machine loses power before the slot opens, a journal (a local append-only log) records "debited, not dispensed." On restart the ATM finishes the dispense or reverses the debit. The journal is the heart of the design. PIN tries: after three failures, keep the session dead and notify the bank. Never store the PIN.

27.8 Vending machine

States: `Idle`, `CollectingMoney`, `Dispensing`, `ReturningChange`. Inventory maps item code to price and count. Inserting money adds to a balance. Selecting an item checks count and balance. Dispensing decrements count. Change is a coin-planning problem like the ATM drawer. Cancel from `CollectingMoney` returns the balance. A jam in `Dispensing` goes to an error state that refunds and does not decrement, or decrements only after the sensor says the item dropped. The sensor is the source of truth for "the user received it."

27.9 Library

Books have copies. Members borrow a copy. Each loan has a due date. A copy is available, on loan, or lost. `borrow` updates the copy from available to on loan inside a conditional update, then writes a loan row in the same transaction. `returnBook` closes the loan, computes a fine strategy from how late it is, and marks the copy available. Reservations: a queue per book. When a copy returns, the next reservation is notified and holds the copy for a limited time. Fine rules are a strategy. Membership limits (five books) are a check inside `borrow`.

27.10 File system

Composite pattern. `Entry` has a name and a parent. `File` is a leaf with size and bytes or a pointer to bytes. `Directory` holds children in a map by name. `add`, `remove`, and `find(path)` live on the directory. Paths split on `/` and walk from the root. A name collision is an error. Deleting a directory deletes children, or refuses if non-empty; pick one and stick to it. Permissions are an optional check object asked before read and write. This design is in-memory. Persisting it is the file-sync HLD: the tree is metadata, the bytes are chunks.

27.11 Snake

A board is a grid. The snake is a deque of cells, head at one end. Each tick, compute the next head from the direction. If the next cell is a wall or a body cell, the game is over. If it is food, push the head and do not pop the tail, and place new food on a free cell. Otherwise push the head and pop the tail. Direction changes queue at most one turn per tick so a quick double tap cannot reverse into the neck. Food placement uses a set of occupied cells for an O(1) membership test. The game object exposes `turn(direction)` and `tick()`. Rendering is a separate observer.

27.12 Chess, the shape

A board is 8 by 8 squares holding a piece or empty. Each piece class implements `legalMoves(board, from)`. A move is from, to, and an optional promotion. The game checks: the piece belongs to the side to move, the destination is in `legalMoves`, and the move does not leave your king in check. Check is "any opposing legal move attacks the king's square," with a careful rule to avoid infinite recursion: the attack test uses raw moves, not the full "does this leave me in check" filter. Special moves are data: castling requires empty squares, unmoved king and rook, and no check along the way. En passant and promotion are explicit

branches. History is a list of moves so you can undo. The design is the piece interface plus the check filter. You do not need a class per rule buried in one giant method.

27.13 LRU cache class

```
Node
  key, value, prev, next

LruCache
  capacity, map, head, tail
  get(key)
    missing -> return empty
    else move node to head, return value
  put(key, value)
    if key exists: update value, move to head
    else: insert at head, map it
          if size > capacity: remove tail from list and map
```

Head and tail can be sentinel nodes so the empty-list cases disappear. One mutex wraps `get` and `put` if the cache is shared. Eviction policy as a strategy is the extension: LFU replaces the list with frequency buckets. The service that uses the cache depends on a `Cache` interface, not on this class.

27.14 In-memory rate limiter

```
TokenBucket
  capacity, refillPerSecond, tokens, lastRefillNanos

  tryTake(now)
    refill(now)
    if tokens >= 1: tokens -= 1; return true
    return false

Limiter
  buckets: map of key -> TokenBucket
  allow(key, now)
```

Refill computes elapsed time and adds tokens, capped at capacity, then sets `lastRefillNanos = now`. The map and the bucket update share a lock per key, or one lock if traffic is modest. In the HLD, this same math runs as an atomic script in a shared cache. The LLD is the math and the state. A clock plug makes tests move time without sleeping.

27.15 Pub-sub inside one process

```
Bus
  subscribers: map of topic -> list of handlers
  subscribe(topic, handler)
  publish(topic, event)
    for handler in copy of list: handler(event)
```

Iterate a copy so a handler can unsubscribe without disturbing the loop. Decide whether handlers run on the publisher's thread or on a pool. If they run on the publisher's thread, a slow handler blocks the publisher. If they run on a pool, order is weaker and errors stay off the publisher. Document that choice. Across processes, replace the bus with the queue from the HLD. The interface `publish(topic, event)` can stay.

28. Checklists, script, and a study plan

The ten-line script

1. Users can do these actions. I am leaving these other actions out of this version.
2. The hot path is this action. Reads and writes compare like this. Data lives for this long. This data may lag. This data must not.
3. Assumptions: these arrivals, these bytes. Result: this many writes per second, this many reads, this much disk. The pressure is on this box.
4. The menu of calls, including the error codes that matter.
5. The tables or documents, the primary key, and the index the hot path uses.
6. The boxes: client, load balancer, stateless servers, source of truth, and any cache, queue, closet, or CDN.
7. The path the user waits on, in order. Then the work that happens after the answer.
8. The first thing that breaks, and the single addition that removes it.
9. Inside the main program: controller, service, repository, and the plugs. The hot method, line by line.
10. Edges: retry, expiry, two writers, a dead dependency, and the state machine if the thing has a lifecycle.

HLD checklist

- Functional list and an out-of-scope list
- Non-functional list: latency, availability, durability, privacy, and allowed staleness
- QPS, storage, and bandwidth, each with its assumption
- API, idempotency, pagination, and errors
- Source of truth, keys, and indexes
- Derived copies: cache, search, feed, CDN, and how each is rebuilt
- Shard key chosen from the hot lookup
- Replication and what a failover can lose
- User path versus queue path
- Timeouts, retries, and what is idempotent
- The rate limit and whether a dead limiter fails open or closed
- Authn and authz on the server
- The metric that proves the objective, and the alert
- The failure of the database leader, the cache, and one application server

LLD checklist

- Use cases
- Entities, value objects, and message objects
- Service methods matching the use cases
- Interfaces for storage, clock, ids, and external providers
- State diagram where a lifecycle exists
- Sequence of the hot use case

- Unique keys and conditional updates for races
- Idempotent retries
- Strategy or state objects where rules will grow
- Tests listed: happy, invalid, duplicate, race, dependency down

A study order

1. Chapters 1–6, then explain TinyLink aloud with the script and no notes.
2. Chapters 7–12. Redo TinyLink and add a cache failure, a shard, and a queue promise.
3. Design a rate limiter and a unique-id service on paper.
4. Design chat. Then the photo feed. Then payments. These three cover fanout, storage of bytes, and money.
5. Chapters 22–26. Implement the LRU cache and the token bucket in real code, with tests.
6. Parking lot, movie tickets, and ATM. Draw the state diagram before the classes.
7. One new product a day from the list below, 40 minutes HLD, 20 minutes LLD of the hottest service. Then compare with this book.

Practice products: URL shortener, paste bin, rate limiter, id service, chat, photo feed, video, notifications, search autocomplete, crawler, file sync, payments, metrics, ride matching, hotel booking, collaborative document, leaderboard, news feed ranking, coupon service, inventory checkout.

For each one, the win condition is the same: another person can implement the hot path and can say what happens when the database leader dies.

Mistakes that ruin an otherwise fine design

- Boxes with no numbers. You cannot know whether one database is enough.
- A cache with no rule for staleness and no way to rebuild it.
- A queue consumer that is not idempotent, paired with at-least-once delivery.
- A shard key that does not match the lookup users actually do.
- Check-then-act for a unique name or a last seat, with no constraint and no conditional update.
- Auto-increment ids in a public URL for private data, with no authorization check.
- A payment flow whose only success signal is the browser.
- Two leaders, or a failover story you have not thought about.
- A class that calls a provider, formats HTTP, and computes a fee. Split the reasons to change.
- A design that cannot name the source of truth.

29. Glossary

Term	Meaning
API	The menu of requests a server accepts, with inputs and outputs
Availability	The fraction of requests or time in which the system answers successfully
Backpressure	Slowing producers when consumers fall behind
Bloom filter	A compact structure that says definitely-no or maybe-yes for membership
Cache	A fast copy of data whose source of truth is elsewhere
Cache-aside	The application fills the cache on a miss and invalidates it on a write
CAP	During a network partition, a system emphasizes consistency or availability
CDN	Caches of content near users, in many cities
Circuit breaker	Stops calling a neighbor that is mostly failing, then probes later
Client	The program that starts a request
Compensation	A new action that undoes the effect of an earlier step in a saga
Composition	A class holds helpers and delegates to them
Consistency	How soon, and for whom, a completed write is visible
Consistent hashing	Placing keys on a ring so adding a node moves only a fraction of keys
Cursor pagination	Fetching the next page using the last seen key, not a large offset
Dead letter queue	Where messages go after repeated failures, for a human to inspect
Denormalization	A deliberate extra copy of a fact, to make a read cheaper
Dependency injection	Passing a helper into a constructor instead of constructing it inside
DNS	Turns a name into an address
Failover	A replica taking the leader's job after the leader stops
Fanout on read	Build a feed when the user opens it
Fanout on write	Push a new post into each follower's prebuilt feed
Foreign key	A column that points at another table's primary key
HLD	High-level design. The city map of a system
Hot key / hot shard	One key or one shard that receives a disproportionate share of traffic
Idempotent	Doing the operation twice has the same effect as doing it once
Index	A structure that finds rows without scanning the whole table
Interface	A plug shape: methods without a single implementation
Inverted index	A map from word to the documents that contain it
Isolation	How much concurrent transactions can see of each other
JWT	A signed, readable token carrying claims such as user id and expiry
Latency	How long one operation takes for one caller
Leader	The replica that accepts writes

LLD	Low-level design. Classes, methods, tables, and rules inside a program
Load balancer	Spreads requests across server copies and skips dead ones
Load shedding	Refusing excess work so accepted work can finish
Lock	A permit that only one thread holds during a critical section
Monolith	One deployable program containing many modules
Normalization	Storing each fact once, in the place it depends on the key
Object storage	A store of blobs addressed by key, used for large files
Outbox	A table of messages written in the same transaction as the business change, to be published soon
PACELC	Partition behavior, and the everyday trade between latency and consistency
Partition (network)	A cut that stops some machines from reaching others
Primary key	The unique identity of a row
QPS	Queries, or requests, per second
Quorum	A minimum number of replicas that must agree for a read or a write
Replication	Copying data to extra machines
Replication lag	How far a follower is behind the leader
Repository	The plug through which a service loads and saves entities
Saga	A series of local transactions with compensations
Service	In HLD, a program with an API. In LLD, the class that holds the rules
Shard	A slice of the rows, stored on its own machines
SLI / SLO / SLA	A measurement, the objective for that measurement, and the promise to a customer
Source of truth	The store whose value wins when copies disagree
Stateful	The process remembers needed facts between requests
Stateless	The process can forget between requests. Any copy can serve the next one
Strategy	A swappable class that implements one policy method
Throughput	How many operations complete per unit of time
TTL	Time to live. How long a cache entry or a hold remains valid
WebSocket	A connection that stays open for messages in both directions

A last page you can recite

Every system is a hot path, a source of truth, and a set of copies you can rebuild. The path the user waits on stays short. Slow work goes to a queue. Retries are safe because the operation is idempotent or carries a token. Unique claims and last seats are decided by a constraint or a conditional update, not by a hopeful check. Freshness is a choice you name: money and ownership sit high, counts and search sit lower. Inside the program, rules live in a service, storage lives behind a plug, and growing rules become new classes. If you can say those sentences about a product, with numbers, you understand its design.

End of the offline guide. System design, HLD and LLD, from zero.

30. The remaining systems people ask for

These use only tools from earlier chapters. Each section names the part that is new.

30.1 Paste bin

A user posts text and receives a link. Opening the link shows the text. Optional expiry and optional unlisted ids. This is TinyLink with a larger body. Store the body in object storage when it can be big, and metadata in SQL. The id is an unguessable random token when the paste is unlisted, because neighbors must not be scanable. Cache hot pastes. A view counter follows the click-queue pattern. Abuse scanning runs on the queue before the paste is marked public.

30.2 Distributed cache as a product

The cache itself is a system. Clients hash a key to a node with consistent hashing and virtual nodes. Each node holds an in-memory map and talks replication to the next node on the ring so one death does not drop every key. A client that guesses the wrong node is told the right one, or a small proxy tier routes for it. Eviction is LRU inside the node. Persistence to disk is optional: turned on when the cache is allowed to be a durable store, left off when it is a rebuildable shelf. Operations that must be atomic (rate-limit decrement, compare-and-set) run inside the node that owns the key. The hard parts are hot keys, resharding when you add nodes, and the choice of what happens on a miss of a dead node: error, or a stale replica.

30.3 Leaderboard

You need the top scores and a player's rank. A sorted map gives both: score order, and the position of one id. In a database, an index on score answers top-100 and a count of rows with a higher score answers rank, and that count gets expensive. A practical design keeps the top of the board in memory or in a sorted in-memory structure, updates it on each new score, and snapshots it to the database. For a global board with very high write rates, shard by game and time window (today, this week), keep each window's top list, and treat exact rank for a low-scoring player as approximate or as a slower query. Say which ranks are exact. Ties need a second sort key, usually time.

30.4 Ride matching

Job. A rider requests a trip. The system offers it to a nearby driver and records the trip from request to drop-off.

Location. Drivers send location every few seconds. You do not run "distance to every driver" as a table scan. Geohash or a grid cell turns a latitude and longitude into a cell id. An index maps cell id to the drivers currently in it. A request looks at the rider's cell and the neighboring cells, then computes true distance for that short list. Location updates are a high write rate; store the latest point in a key-value store and keep history only if the product needs it.

Match. A trip row is created in state `requested`. A matcher picks the closest eligible driver who is free and pushes an offer on that driver's socket. The driver accepts or the offer times out and the matcher tries the next. The conditional update is `set state = offered where id = ? and state = requested`, and for the driver `set status = reserved where driver_id = ? and status = free`. Exactly one matcher wins. States then run `requested → offered → accepted → started → completed`, with `cancelled` from the early states.

Hard parts. Location is stale by a few seconds; say so. A surge of requests in one cell is a hot cell: the matcher for that cell must be sharded by finer cells. Payments attach at completion using the payments design. ETA is a call to a routing service, cached briefly, and allowed to be approximate.

30.5 Hotel or inventory booking

Same race as movie seats, different words. A room type on a night has a remaining count, or each physical room is a row. A hold decrements availability with a conditional update `where remaining >= rooms_requested` and writes a hold that expires. Confirm turns the hold into a booking after payment. Overbooking policy, if the business wants it, is an explicit extra count, not an accident of a race. A calendar of nights is the query: index by `(hotel_id, night)`. Searching a city is a read model: a document per hotel with amenities, updated from the source of truth, queried by the search index.

30.6 Flash sale and inventory

A small stock meets a huge burst. The database row "20 left" will serialize all the buyers if every purchase locks it. Practical path: before the sale, place the 20 tokens in a cache or a queue. Each buyer takes one token atomically. Only token holders proceed to checkout and payment. When payment fails, the token returns. When the tokens are gone, the service refuses quickly without touching the database. The database still records the order as the source of truth. The token pile is an admission gate. This keeps the hot second off the SQL row lock. After the rush, reconcile tokens against paid orders.

30.7 Coupons

A coupon has a code, a rule (percent or amount), a validity window, a global redemption cap, and a per-user cap. Redeem is a conditional update: insert a redemption row with a unique `(coupon_id, user_id, order_id)` and increment `used_count` only if `used_count < cap`. Two parallel checkouts for one user lose on the unique key. Rules that stack or exclude each other belong in a strategy object evaluated before the write. The write is what makes the cap real.

30.8 Collaborative document

Several people edit one document and should converge. Two families of rules:

Operational transform. Each edit is an operation, such as "insert 'a' at index 4." The server orders operations and rewrites later operations so they still make sense after earlier ones. A central order is required. This is how many classic co-editors work.

CRDT. Conflict-free replicated data types. Each character has a unique id and a position that sorts the same way on every replica. Inserts and deletes commute, so replicas that receive the same operations in different orders reach the same document. The server can be a relay. Metadata per character is the cost.

The rest of the system is familiar: a WebSocket gateway, a log of operations per document (the source of truth), periodic snapshots so a new joiner does not replay ten years, and authorization on the document. Presence of cursors is a side channel and may be lossy. Offline edits are operations queued on the client and merged when the client reconnects, using whichever rule you chose.

30.9 Social graph and recommendations, lightly

"Who should I follow?" is a batch or nearline job, not a query you invent on the tap. A job walks follows, computes simple signals (friends of friends, overlap), and writes the top suggestions per user into a key-

value row. The tap reads that row. Recompute on a schedule and after a follow. Doing the walk live, on a hot path, does not survive popular users.

30.10 Counters at high speed

A like button pressed millions of times will crush a single row if every press is a transaction on it. Buffer increments in a cache, flush `add N` to the database every second, and accept that the displayed number lags. If the user must see their own like immediately, store the user's like as a row (unique on user and post) and let the displayed total be the lagged counter. The unique row is the source of truth for "did I like this?" The counter is a derived display.

30.11 Backups and deploys

Replication is not a backup. A mistaken delete replicates. Take snapshots on a schedule, keep the database log so you can restore to a point in time, and practice a restore. For files, object storage versioning or a copy to a second region serves the same role.

Deploys should allow an old server and a new server to run together. That means the database change goes first and stays compatible with the old code: add a column before you write it, stop reading the old column before you drop it. Rolling deploy replaces servers a few at a time behind the load balancer. A health check fails the new server out of the pool if it cannot talk to the database.

30.12 Connection pools, timeouts, and budgets again

Each application server holds a small pool of database connections and reuses them. If you open a connection per request, the database spends its life on handshakes and then refuses new ones. Size the pool so that servers × pool size stays under the database's max connections, with room for replicas and admin. When the pool is exhausted, fail the request. Waiting forever is how a slow database takes down every server.

Give the user request a deadline. Subtract time already spent before each hop. Pass the remaining budget along. A hop that cannot finish inside its slice is cancelled.

31. How HLD and LLD meet on one page

HLD box or promise	What LLD must show
Stateless API server	No user session stored in a field of a long-lived singleton. Request data arrives as arguments
SQL unique code	Repository insert surfaces a conflict. Service turns it into a result, and does not assume the pre-check is enough
Cache-aside	`Cache` plug with get, put, delete. Service defines miss, expiry re-check, and what happens when the cache throws
At-least-once queue	Consumer stores the message id with the effect, or the effect itself is an idempotent add that you have consciously allowed to be approximate
Idempotency key	A unique column and a lookup-before-or-during insert that returns the original result
State machine for payment or booking	An enum of states and the only legal transitions. Illegal ones return a clear error
Rate limit	A pure function of tokens and time, plus an atomic update in the shared cache
Authorization	A check inside the service or a plug called before the rule runs, using the caller id from the token, not a flag from the client
Failover loses the last async writes	The LLD names which operations the user may need to retry, and which tokens make that retry safe

When an HLD sentence has no partner in the LLD, the design still has a hole. When an LLD class has no reason in the HLD, it is decoration. Keep the pairs.

32. A compact formula sheet

```
QPS ≈ events_in_period / seconds_in_period
peak QPS ≈ average QPS × peak_factor

storage ≈ rows × bytes_per_row × replication_factor × index_factor
index_factor is often 2 or more

bandwidth ≈ QPS × bytes_per_response
megabytes/s ≈ (QPS × bytes) / 10^6

cache size ≈ hot_keys × value_bytes × overhead

availability downtime per year
  99.9%   ≈ 8.8 hours
  99.99%  ≈ 53 minutes
  99.999% ≈ 5 minutes

quorum overlap: R + W > N

base62 space of length L: 62^L
  L = 6  ≈ 5.7 × 10^10
  L = 7  ≈ 3.5 × 10^12

snowflake layout, one common choice
  1 unused + 41 time + 10 machine + 12 sequence
  sequence per machine per millisecond: 4096

token bucket
  tokens = min(capacity, tokens + elapsed_seconds × refill_per_second)
  allow if tokens >= 1, then tokens = tokens - 1
```

33. What this book deliberately leaves to practice, not to mystery

You now have the concepts, the checks, and a worked design for every standard shape: links, limits, ids, chat, feeds, files, video, search, crawl, payments, metrics, rides, bookings, sales, coupons, documents, caches, counters, and the class drills. Vendor names change every year. The behaviors in this book do not. When you meet a new product, run the ten-line script. If a step has no answer, that step is the thing to design, and the catalog chapter for that step is the chapter to reread.

Implement two of the LLD drills in real code so the words become fingers: the LRU cache, and the movie-ticket hold with a conditional update. Then design one system a day on paper for two weeks. The offline test is whether you can fill the checklists in chapter 28 without opening this file, and then open it only to see what you skipped.